

CG e-Procurement Chatbot

Project Architecture & Deployment Guide

An AI-powered Retrieval-Augmented Generation (RAG) assistant designed for the **Chhattisgarh e-Procurement Portal**. The chatbot helps users (Vendors/Contractors and Government Officers) query procurement manuals, General Financial Rules (GFR), and CVC compliance guidelines in English, Hindi, and Hinglish.

■ Key Features

- **Dual-Language & Transliteration Pipeline:** Supports search queries in English and Hindi.
 - Queries in Hindi are translated to English dynamically using Google Translate to fetch the most relevant rules.
 - English responses from the LLM are translated back to Hindi (Devanagari script) using a local, CPU-optimized Indic translation model (sarvam-cpu).
 - If Hinglish is detected, Devanagari Hindi is dynamically transliterated into conversational Roman script (Hinglish) at the end of the streaming pipeline.
- **Overlapped Pipelined Streaming:** Multi-threaded parallel translation loop. While a background thread streams English generation from the local LLM, the main thread translates completed sentences on-the-fly, reducing the user's perceived latency.
- **Conversational Memory:** Context-aware follow-ups. Short or vague follow-up queries (e.g. "next step kya hai?") are semantically rewritten based on the recent conversation history to retrieve the correct contextual chunks.
- **Interactive PDF Viewer & Citations:** Every answer cites its source rules and document references (e.g. [Page 42]). Clicking the citation chip opens the PDF exactly on the cited page with the matched terms highlighted.
- **Interactive UI Actions:**
 - **Text-to-Speech (TTS):** Play answers out loud in Hindi or English.
 - **Copy to Clipboard:** Copy plain-text answers with markdown stripped.
 - **Muted Footnote Disclaimer:** Subtle footnote indicating that the response is AI-generated for reference only.

■ Architecture

Module / Component	Technology / Model
Backend	FastAPI (Python 3.11)
Frontend	React + Webpack (interactive chat widget)
Vector Store	Qdrant Database (separate collections: vendor_manuals & govt_rules)
LLM Engine	Fine-tuned Llama 3 8B / Qwen-2.5-3B (running locally via Ollama)
Translation Model	sarvam-cpu (Indic machine translation, running locally via Ollama)
Embeddings Model	bge-m3 / nomic-embed-text (running locally via Ollama)

■ Project Setup

1. Prerequisites

Ensure you have the following installed on your machine:

- Python 3.11
- Node.js (v18+)
- Ollama

2. Ollama Models

Download the required models in Ollama before starting:

```
ollama pull mistral:latest
ollama pull qwen2.5:3b
ollama pull nomic-embed-text:latest
# For Hindi translation:
ollama pull mashriram/sarvam-1:latest
```

3. Backend Setup

Navigate to the root directory and create a Python virtual environment:

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the document ingestion script to parse PDFs and populate the Vector Store:

```
venv\Scripts\python.exe scripts/ingest_documents.py
```

Start the FastAPI backend server:

```
venv\Scripts\python.exe -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

4. Frontend Setup

Navigate to the frontend folder, install packages, and launch development build:

```
cd frontend
npm install
npm run dev
```

■ Evaluation & Testing

The repository contains automated testing suites to evaluate retrieval and response accuracy:

- **Retrieval Recall Test:** Evaluates Top-1, 3, 5, and 10 retrieval recall across 100 benchmark queries targeting complex PDFs.

```
python scripts/evaluate_rag.py
```

- **End-to-End KPI Test:** Uses LLM-as-a-judge to grade chatbot accuracy against a set of 80 ground-truth Q&As;

```
python scripts/run_kpi_testing.py
```